

מעבדה 5: ניהול זיכרון בשיטת רשימות מקושרות

הקדמה ומטרות

במעבדה זו נממש את אחד המנגנונים המרכזיים בניהול זיכרון של מערכת הפעלה: הקצאת זיכרון רציפה (Contiguous Memory Allocation) נלמד כיצד לנהל רשימה של מקטעים (Segments) שהם או "תהליכים" או "חורים", וכיצד אלגוריתמים שונים משפיעים על ניצולת הזיכרון.

מטרות המעבדה

- מימוש מבנה נתונים של רשימה מקושרת לייצוג מפת הזיכרון.
- מימוש אלגוריתמי הקצאה *First Fit* & *Best Fit*
- הבנת תהליכי פיצול (*Splitting*) ומיזוג (*Coalescing*) של זיכרון.

מבנה הנתונים והקבצים

הזיכרון ייוצג על ידי רשימה מקושרת של צמתים. כל צומת (Segment) מייצג בלוק זיכרון רציף.

עליכם לממש את מבנה הנתונים הבא (נתון בקובץ *memory_manager.c*)

```
typedef struct Segment {
    int is_process;      // אם הבלוק תפוס ע"י תהליך, 0 אם הוא חור פנוי 1
    int process_id;      // מזהה התהליך (במידה ותפוס, אחרת -1)
    int start_address;   // כתובת התחלה של הבלוק
    int size;            // גודל הבלוק
    struct Segment *next;
} Segment;
```

דגשים והסברים

אלגוריתמי ההקצאה (*Allocation*)

כאשר מגיעה בקשה להקצאת זיכרון בגודל S

- First Fit** סורקים את הרשימה מתחילתה ובחרים בחור הראשון שגודלו $S \leq$ (גדול שווה ל S)
- Best Fit** סורקים את כל הרשימה ובחרים בחור שגודלו $S \leq$ אך הוא הקטן ביותר מבין כל האפשרויות (כדי לשמור חורים גדולים לבקשות עתידיות).

פיצול ומיזוג (*Splitting & Coalescing*)

- פיצול**: אם מצאנו חור שגודלו 50 יחידות והתהליך דורש 30, נהפוך את הצומת לצומת תפוס בגודל 30, וניצור צומת חדש (חור) בגודל 20 מיד אחריו.
- מיזוג**: בעת שחרור תהליך, הסגמנט הופך לחור ($is_process = 0$). מיד לאחר מכן, יש לבדוק אם השכן מלפניו או מאחוריו הוא גם חור. אם כן, יש לאחד אותם לצומת אחד גדול כדי למנוע פרגמנטציה.

משימות לביצוע

משימה 1: אתחול והדפסת מפת הזיכרון

- ממשו הפונקציה *init_memory* המאתחלת את הזיכרון בגודל 1024 יחידות, כחור אחד גדול.
- ממשו את הפונקציה *print_memory* המדפיסה את מצב הזיכרון כך

$[P1, Addr: 0, Size: 10] \rightarrow [H, Addr: 10, Size: 90] \rightarrow NULL$

עבור כל צומת (*Segment*) ברשימה, נדפיס את סוג המקטע, כתובת ההתחלה והגודל שלו בתוך סוגריים מרובעים. בין המקטעים נדפיס חץ ($\rightarrow$) כדי לסמן את הקשר הלוגי ברשימה.

- **P (Process)** מייצג מקטע תפוס.
- **H (Hole)** מייצג מקטע פנוי (חור).
- ממשו את הפונקציה `print_stats` המציגה את ניצולת הזיכרון הכוללת. פורמט הפלט לסיכום:

```
--- Memory Statistics ---
Total Memory: 1024 units
Used Memory: 450 units (43.9%)
Free Memory: 574 units (56.1%)
Number of Segments: 5 (3 processes, 2 holes)
-----
```

משימה 2: Allocation Engine

ממשו את הפונקציות `allocate_first_fit` ו `allocate_best_fit`

- זכרו לבצע פיצול (*Splitting*) במידת הצורך.
- אם אין חור מתאים, יש להדפיס הודעת שגיאה לערוץ השגיאות. *Out of Memory*.

משימה 3: Deallocation & Coalescing

ממשו את הפונקציה `deallocate_mem(Segment * head, int process_id)`

- הפונקציה תמצא את התהליך ותהפוך אותו לחור.
- **חובה:** לבצע מיזוג (*Coalescing*) עם חורים סמוכים מיד לאחר השחרור.

משימה 4: Interaction

עליכם להשלים את הקובץ `main_memory.c` בקומות המסומנים.

השתמשו ב `main` המסופק כדי להריץ תרחישים ולבדוק:

האם ב *Best Fit* נשארו חורים קטנים יותר מאשר ב *First Fit*

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם `Lab5_ID.zip` (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. `main_memory.c`

2. `memory_manager.c`

3. `memory_manager.h`

4. `commands.txt`

5. `Makefile`

דגשים להגשה:

- יש לוודא שלכל הקבצים יש הרשאות ריצה. (`chmod +x`)

- הבדק האוטומטי משווה פלטים בצורה מדויקת (Case Sensitive) אין להדפיס הודעות נוספות מעבר לנדרש.
- וודאו שכל סקריפט מתמודד נכון עם ארגומנטים חסרים.

אי הקפדה על הנחיות, כולל פורמט הגשה הדיגיטלי, תגרור הורדה בציון התרגיל.